

Q20 and vol connections m8.

MP Neuron & Perceptron

Biological Neuron has 3 parts

- Dendrites: Receives signals from other neurons.
- Soma: Processes the info
- Axon: Transmits the op
- Synapse: Connects with other neuron.

Neuron is activated

Neuron is activated when its criteria is met:

Neurons are arranged in a hierarchical order and each layer has its own role or significance.

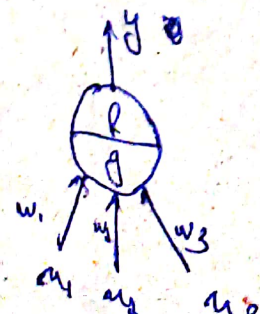
First step towards perception

1943 McCulloch & Pitts Neuron

It mimics the functionality of a biological neuron. Artificial Neuron:

The fundamental building block of deep learning

Artificial Neuron Network



This is the first computational model of neuron.

g takes inputs, performs aggregation based on aggregated results. f makes a decision

$$g(m) = m_1 + m_2 + \dots + m_n = \sum_{i=1}^n m_i$$

$$y = f(g(m))$$

~~Thresholding~~

Threshold logic: is used to mimic the thought process:

$$y = 1 \quad \text{if} \quad g(m) \geq 0$$

$$y = 0 \quad \text{if} \quad g(m) < 0$$

There can be 2 types of inputs (m_i):

Inhibitory inputs - It has maximum effect on decision making irrespective of other inputs.

Excitatory inputs - Doesn't make neuron fire on its own.

Loss function

We aim to minimize the loss function.

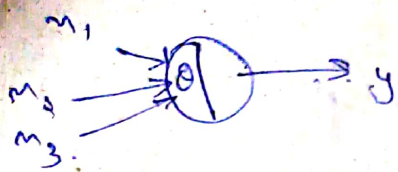
$$\text{Loss} = \sum_i (y_i - \hat{y})^d$$

$$\hat{y} = m_1 + m_2 \geq b$$

Accuracy of model

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Boolean function using MP Neuron



Activation
 $g(m) \geq 0$



$g(m) \geq 3$



$g(m) \geq 1$

Limitation of MP Neuron

- * Inputs cannot be real
- * No importance to particular input.
- * Threshold has to be hardcoded
- * Cannot do decision making for Non linearly separable data

1958 Perceptron Model Frank Rosenblatt

- * Weights & threshold can be learnt over time.

- (i) Perceptron model
- (ii) Geometric interpretation.
- (iii) Learning algorithm

Perceptron model is similar to that of MP Neuron but it takes weighted sum of inputs & sets the output 1 if the sum is more than a threshold $\text{threshold}(0)$

To learn θ , we take $w_0 = -\theta$ with $w_0 = 1$

is $y = 1$ if $\sum_i w_i m_i - \theta \geq 0$

$y = 0$ if $\sum_i w_i m_i - \theta < 0$

(here θ is called bias)

to make it learnable.

- * The weight & the bias will depend upon the data.

Geometrical Interpretation

MP Neuron

$$\hat{y} = m_1 + m_2 \geq b$$

$$m_1 + m_2 \geq b$$

$$m_2 \geq -m_1 + b$$

$$m \geq -1 \quad c \geq b$$

Fixed slope

Perceptron

$$\hat{y} = w_1 m_1 + w_2 m_2 \geq b$$

$$m_2 \geq \frac{-w_1 m_1 + b}{w_2}$$

$$m_2 \geq \frac{-w_1}{w_2} m_1 + \frac{b}{w_2}$$

$$m \geq \frac{-w_1}{w_2} \quad c \geq \frac{b}{w_2}$$

Flexible slope

Dot product concept in perceptron

$$W = [w_0, w_1, \dots, w_n]$$

$$x = [1, x_1, \dots, x_n]$$

$$\text{dimension} = n + 1$$

$$W \cdot x = W^T x = \sum_{i=0}^n w_i x_i$$

This dot product quantifies how much one vector goes in the direction of the other.

The decision boundary that perceptron gives is

$$W \cdot x \geq 0$$

$$W \cdot x = W^T x = \|W\| \|x\| \cos \alpha$$

$$\cos \alpha = \frac{W^T x}{\|W\| \|x\|}$$

gives the angle b/w the vectors.

Perceptron Learning Algorithm

Initialize weights.

Iterate over data:

$L = \text{compute_loss}(m)$

update (weights, L)

tilt satisfied.

when x belongs to P we want $W \cdot x \geq 0$ i.e. angle b/w W & x should be less than 90° .

Initialize w randomly
while !converge do

 Pick random x .

 if $x \in P$ & $\sum_i w_i x_i < 0$

$$w = w + x$$

end

 if $x \in N$ & $\sum_i w_i x_i \geq 0$

$$w = w - x$$

end

end.

Sigmoid neuron

- * smoothest activation function
- * S-shaped function curve
- * Monotonic
- * Differentiable (i.e. we can find slope at any two points)
- * Mostly used for prediction.

$$y = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

σ is called the sigmoid function.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

A monotonic function is either non-increasing or non-decreasing.

Gradient descent is calculating $\hat{y}$ of sigmoid function

LEARN

UAT & Multi-layer Perceptron

In real life the relationship b/w input & output will be complex, unlike perceptron or sigmoid neuron.

∴ we have to find a complex function that represents this relation.

Universal approximation theorem

We can always come up with a DNN that can approximate a complex relation b/w input & output.

OR

A FFN with a single hidden layer with finite No of neurons can approximate arbitrary functions.

* To solve a complex function we can just come up with a single function & use the combination of these functions to approximate the relationship to any true relationship.

~~How to come up~~

Towers.

Input x is passed through two sigmoid neurons & combined in other neurons with weights $+1$ & -1 (same as subtracting their output); then we get our towers.

This is called as the representational power of DNN, to approximate complex relationship b/w input & output.

Feed forward Neural Network FFN

* The connections b/w nodes do not form a cycle.

Multiple sigmoid neurons

$$h_1 = f_1(m_1, \dots, m_2)$$

$$h_2 = f_2(m_1, \dots, m_2)$$

$$\hat{y} = g(h_1, \dots, h_2)$$

$$h_1 = \frac{1}{1 + e^{-(w_{11}m_1 + w_{12}m_2 + b_1)}}$$

$$h_2 = \frac{1}{1 + e^{-(w_{21}m_1 + w_{22}m_2 + b_2)}}$$

$$\hat{y} = \frac{1}{1 + e^{-(w_{31}h_1 + w_{32}h_2 + b_3)}}$$

Pre activation of layer i

$$a_i(m) = w_i h_{i-1}(m) + b_i$$

Activation of layer i

$$h_i(m) = g(a_i(m))$$

Hyperparameter tuning

- (i) Gradient descent
- (ii) Types of gradient descent
- (iii) Algorithms

Gradient descent

• Gradient descent is a way to minimize the objective function.

• Objective function is parameterized in the opposite direction of its gradient.

$$\begin{aligned} w_{t+1} &= w_t - \eta \Delta w_t \\ b_{t+1} &= b_t - \eta \Delta b_t \\ \Delta w_t &= \frac{\partial}{\partial w} L(w, b) \\ \Delta b_t &= \frac{\partial}{\partial b} L(w, b) \end{aligned}$$

η is the learning rate which determines the size of the steps we take to reach the local minimum.

Steps

- (1) Compute the slope i.e. 1st derivative at that point
- (2) Move in the opposite direction of slope until we reach minimum point where slope is zero.

Depending on the amount of data we make a trade off b/w the accuracy of no parameter update and time it takes to perform update

1-step: one update of the parameters.

1-epoch: one pass over the entire dataset.

N: No of data points

B: miniBatch size

Algorithm	# steps in one epoch
Batch GD	1
stochastic Stochastic GD	N
mini-batch GD	N/B

Batch GD

~~stochastic~~ Stochastic GD

mini-batch GD

Batch gradient (1 step in 1 epoch)

All training data is considered to take one single step

Gradient at every data point is taken & its mean gradient is used to update parameters. (we move already towards set)

Stochastic GD

In stochastic GD we just consider one example at a time to take a step

- * The cost will not necessarily decrease
- * Because the cost is so fluctuating it will never reach minima
- * SGD can be used in case of a larger dataset as it converges faster since it causes updates to the parameter map frequently.

Mini batch gradient descent
(N/B steps in 1 epoch)

Batch GD

Smoothen curve
converges directly to minima

SGD

large dataset
converges faster for large dataset

After creating mini batch of fixed size, it iterate over minibatch of size B
N/B steps in 1 epoch

Normal batch size: 32, 64, 128

MBGD has smoothen cost function than SGD

Issues with GD (vanilla GD)

- * Takes lot of time to navigate regions having gentle slope

Momentum Based GD

Concept: If we are moving in a direction multiple time we should gain the confidence & take bigger steps.

$$\begin{aligned} w_t &= w_t - V_t \\ V_t &= \mu \Delta w_t + \gamma V_{t-1} \end{aligned}$$

the current update is not just proportional to the present gradient but also to the previous gradient.

Advantages & Dis of MBGD

It oscillates in & out of minima valley as the momentum carries it out of the valley.

Momentum cause overshoot & ~~causes~~ results in lot of U-turns because converging.

But still ~~fast~~ faster than vanilla GD

NAG Nesterov Accelerated GD (NAG)

Look before you leap.

- * It takes place in two steps
- (i) get the look ahead point.

$$\cancel{w_{temp} = w_t + \eta V_{t-1}}$$

$$w_{temp} = w_t - \eta V_{t-1}$$

- (ii) final update which is negative

$$w_{t+1} = w_t - \eta \Delta w_{temp}$$

$$V_t = \eta \Delta w_{temp} + V_{t-1}$$

- * still results in overshooting but a smaller one compared to MBGD

why adaptive learning rate is required?
→ for sparse data (η should be aggressive)

when feature is on less frequently then learn aggressively when it's on

→ For dense data (η should be less aggressive)

Ada grad (Adaptive gradient optimiser)

- * Parameters corresponding to sparse features get better updates.
- * Learning rate decays aggressively as denominator grows (not good for dense features)

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{V_t} + \epsilon} \Delta w_t$$

$$V_t = V_{t-1} + (\Delta w_t)^2$$

RMS Prop

- * adapts get stuck when close to convergence
- * RMS Prop overcomes this by being less aggressive.

Concept: Decay the denominator.

$$V_t = \beta V_{t-1} + (1-\beta) (\Delta w_t)^2$$

RMS Prop will prevent blow up of denominator for dense features.

Adam

combines the advantages of momentum & RMS Prop

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{V_t} + \epsilon} m_t$$

$$m_t = \beta_1 V_{t-1} + (1-\beta_1) \Delta w_t$$

$$V_t = \beta_2 V_{t-1} + (1-\beta_2) (\Delta w_t)^2$$

Activation functions

Logistic / Sigmoid

$$y = \frac{1}{1 + e^{-n}} = f(n)$$

$$f'(n) = \frac{d}{dn} f(n) = f(n) \cdot (1 - f(n))$$

* Always normalize input so that it lies b/w 0 & 1

saturation happens when

$$f(n) = 0 \text{ or } 1$$

$$f'(n) = 0$$

if saturated neurons causes no gradients to vanish.

no learning happens

* Not zero centered.

Tanh

* zero centered.

disadv

* computationally expensive (e^n)
* saturated neurons cause vanishing gradient.

$$f(n) = \frac{e^n - e^{-n}}{e^n + e^{-n}}$$

$$f'(n) = 1 - (f(n))^2$$

ReLU (Rectified Linear unit)

- * Not zero centered.
- * ~~saturated neurons~~ Does not saturate in +ve region
- * Computationally not expensive.

$$f(n) = \max(0, n)$$

$$f'(n) = \begin{cases} 0 & \text{if } n \leq 0 \\ 1 & \text{if } n > 0 \end{cases}$$

Issue with ReLU activation

- * if the learning rate is really high Δb can be highly -ve

$$b_1 = 0$$

This causes large fluctuations of ReLU units to die.

→ Advised to keep learning

~~rate~~ +ve
bias +ve

Leaky ReLU

* ~~zero centered~~

- * Computationally not complex
- * Does not saturate in both +ve & -ve ~~direction~~ region.

* ~~units~~ units will not die.

$$f(n) = \max(0.01n, n)$$

$$f'(n) = \begin{cases} 0.01 & n < 0 \\ 1 & n > 0 \end{cases}$$

0.01 ensures that no units don't die.

underfitting: Model fails to learn the underlying pattern in the data.

overfitting: Model learns the underlying pattern as well as noise.

Bias: Difference b/w avg prediction of model & actual value

$$\text{Bias}(\hat{f}(n)) = E[\hat{f}(n)] - f(n)$$

* High error in both training & testing

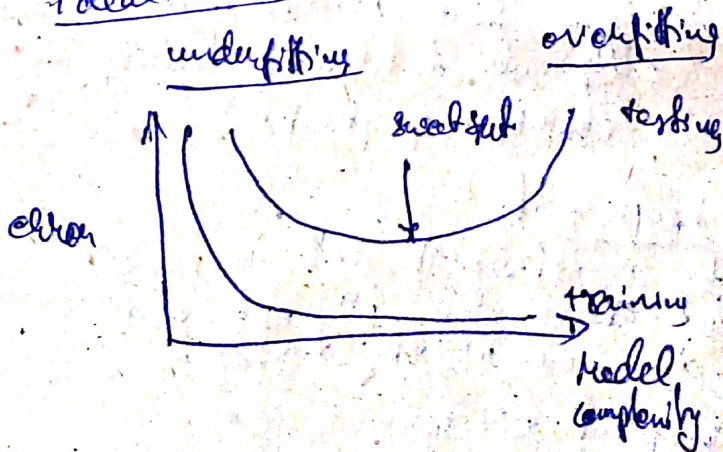
Variance: Variability of model prediction for a given data point.

* spread of the data
* High error in testing.

Simple model: high bias, low variance

Complex model: low bias, high variance

ideal model: low bias, low variance



has variance made off

- * Split dataset in Train , Valid
- * choose
 - (i) ~~the~~ right activation function
 - (ii) right initialization method (i.e. Xavier)
 - (iii) Monitor training & validation error.

Solution for underfitting:

- (i) Increase the complexity of function
- (ii) Train for more epochs.

Solution for overfitting:

- (i) More training data (data augmented)
- (ii) Regularization
- (iii) Early stopping (~~loss~~ train loss)

Xavier initialization - $\tanh$ & tanh activation

initializes weight to lie in a uniform distribution of $\left[\pm \frac{\sqrt{6}}{\sqrt{n_i + n_{i+1}}} \right]$

$n_i \rightarrow$ # incoming network connections
 $n_{i+1} \rightarrow$ # outgoing network connections
($n_{in} - n_i$) ($n_{out} - n_{i+1}$)

He initialization with relu activation

* Calculates a random number with gaussian distribution probability (G) with mean 0 & standard deviation $\frac{1}{\sqrt{n}}$

$n \rightarrow$ no of inputs to node

Data Augmentation

- * Techniques to generate different versions of real data artificially to increase its size.

eg: Translation : flipping, cropping
Scaling, Rotation.

Regularization

- * Regularization is a technique that ~~penalizes~~ punishes the coefficients. In overfitted model, coefficients are inflated.

- * The coefficients are added to a cost function of linear equation.

∴ Linear regression model will optimize coefficients to minimize cost function.

Early stopping

Stopping criteria: Training is stopped as soon as validation error increase w.r.t previous epoch.

Batch normalization

Batch norm is a normalization technique done on b/w layer of NN instead of raw data.

- * It is done along mini batches
- * It speeds up the learning & uses high learning rates to make learning easier.

$$h_{ij}^{norm} = \frac{h_{ij} - \mu_j}{\sigma_j}$$

μ_j is the mean of all the neuron output. & σ_j the corresponding std. deviation

$$\mu_j = \frac{1}{m} \sum_{i=1}^m h_{ij}$$

$$\sigma_j = \sqrt{\frac{1}{m} \sum_{i=1}^m (h_{ij} - \mu_j)^2}$$

Ensemble method (like RF model)

- * Relies on multiple model than a single model.

Ensemble in Neural network

- (i) Train diff. model with same dataset. (expensive)
- (ii) Train same model with different subset of training dataset (expensive)

Drop out

- * for a network with n nodes we can create 2^n networks with some weights dropped.

Drop out criteria

Randomly assign probabilities to nodes. Randomly drop out drop out nodes with probability

Back propagation : Algorithm
designed to test for errors
working back from output
to ~~input~~ nodes to input node
* It improves accuracy.

Algorithm for training

- (i) Initialize Parameters.
 - (ii) it create over data
 - * choose minibatch
 - * choose drop out network.
 - * calculate loss
 - * Back propagation.
- if satisfied.

using drop out at testing

• In testing scale up the
output of each neuron by P

Convolution neural network

Convolution: Mathematical way of combining two signals to form a third signal

2-D convolution

Convoluting an input of $n \times n$ dimension with a $k \times k$ filter results in $(n-k+1) \times (n-k+1)$ size output.

3-D convolution

Input 3D, filter 3D $\rightarrow$ output = 2D

* This is because the convolution operation we are performing are 2D

* Each filter applied to a 3D input will give 2D output

* Combining the output of multiple such filters will result in a 3D output.

Terminologies to calculate W_o, H_o, D_o

$F \rightarrow$ Spatial extent of a filter

$k \rightarrow$ # of filter

$P \rightarrow$ Padding

$S \rightarrow$ stride

No padding (size of output reduces)

$$W_o = W_i - F + 1$$

$$H_o = H_i - F + 1$$

$$D_o = k$$

i.e. each filter gives 2D output

$\therefore k$ such outputs will be there.

$$D_o = k$$

with padding

$$\text{Input} = n \times n$$

$$\text{padding} = P$$

$$\text{Filter size} = k \times k$$

$$O/P = \frac{(n+2P-k+1)}{S} \times \frac{(n+2P-k+1)}{S}$$

Strided convolution

Stride is a parameter that defines the movement of kernel/filter over input image.

$$O/P = \frac{n-k}{S} \times \frac{n-k}{S}$$

$$O/P = \frac{(n+2P-k+1)}{S} \times \frac{(n+2P-k+1)}{S}$$

* stride helps to reduce the size of the image.

Q) $w_i = 7$, $H_i = 7$
 $F = 3$, $P = 1$, $S = 2$

$$w_o = \left(\frac{w_i + 2P - F}{S} \right) + 1$$

$$= \left(\frac{7 + 2 - 3}{2} \right) + 1$$

$$= 3 + 1 = 4$$

$$H_o = \left(\frac{7 + 2 - 3}{2} \right) + 1 = 4$$

$D_o = 16$

Pooling: Pooling layers are used to reduce the size of the inputs to speed up computation.

Max pooling: It is a convolution process where the kernel extracts the ~~maximum~~ maximum value of the area it convolves.

Helps in extracting low-level features like points & edges

HOG (Histogram of gradient)

- (i) splits the image into different regions
- (ii) finds the magnitude & direction of edges in each region.
- (iii) draws histogram of each of the region separately

SIFT (Scale invariant feature transform)

- (i) constant scale space
 - (ii) key point localization
 - (iii) orientation assignment
 - (iv) key point descriptor & unique fingerprint of each key point
- scale invariant, Rotation invariant.

In CNN ~~there~~ there are three types of layers

- (i) convolution layers
- (ii) pooling layers
- (iii) fully connected layers.

Transfer Learning Pre Trained model

* People usually don't train a CNN from scratch.

* ~~It is a~~ First we pre-train a CNN on a very large data set & use it for initialization or feature extraction

Steps:

- (i) Take a pre-trained CNN model
- (ii) Freeze the parameters of the lower convolution layers
- (iii) Replace the fully connected layer with a custom classifier layer with trainable parameters.
- (iv) Train the classifier layer with the training data
- (v) Perform hyperparameter tuning & unfreeze the parameters as needed.

Back propagation: Algorithm designed to test for errors working back from output to ~~input~~ nodes to input node.
 * It improves accuracy.

Algorithm for training

- (i) initialize parameters.
 - (ii) it create over slots
 - * choose minibatch
 - * choose drop out network.
 - * calculate loss
 - * back propagation.
- it satisfied.

using drop out at testing

in testing scale up the output of each neuron by P

Receptive field

check what is causing the neuron to fire.

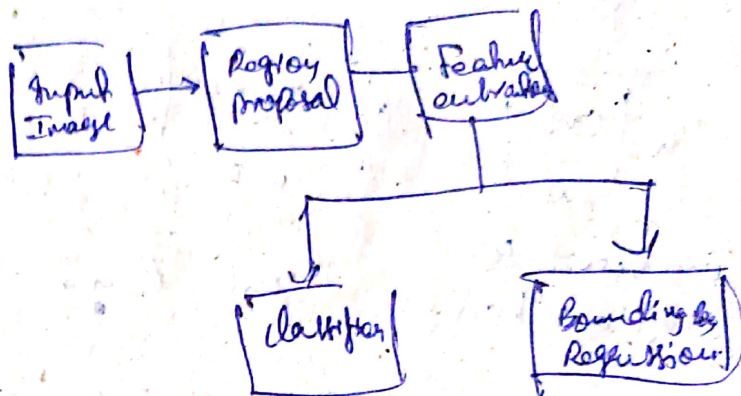


Object detection

For object detection we should find the object as well as draw exact bounding box over the object.

It is proposed as a region proposal & bounding box regression problem.

R-CNN



Region proposal - selective search

- (i) Generate sub-segmentations to get candidate regions
- (ii) use greedy algorithm to recursively combine similar region to largest ones.
 [Hierarchical clustering at different scale]
- (iii) Generated region is given as Region proposal.

(b) Proposed regions are cropped to form mini images
 c) They are scaled to fit CNN's input size.

d) Feature extraction is done using transfer learning

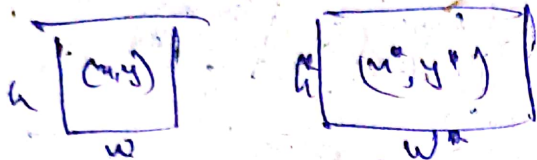
e) SVM is used for classification

$$\min \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{w} - w_i \hat{z}_i \right)^2$$

$$\min \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{w} - \log \hat{z}_i \right)^2$$

here $\frac{m^* - m}{w}$ is the normalized difference b/w m & m^* .

The model predicts w, T as this difference.



~~Predicted Bo~~
Proposed bo

True Bo.

Disadvantages of R-CNN

- * Take large amount of time b/c we have to classify lot of Region proposals.
- * Takes 47 sec for each test image $\therefore$ real time implementation is not ideal
- * Selective search is fixed, no learning. could lead to bad region proposals during generalisation.

Faster R-CNN

- * It is faster b/c we don't have to feed 2000 region proposals to CNN. Instead the convolution operation is done once per image & a feature map is generated from it.

- * From the feature map identify Region proposals
- * wrap them into squares
- * Pooling to resize to image to feed it to fully connected layers.
- * RoI feature vector contains soft max layers which can classify proposals as well as predict bounding box offset values.

YOLO (You Look ONLY once)

- * It takes entire image in a single instance & predict the bounding box coordinates and class probabilities for each box.
- 45 FPS with generalized object representation.